Computational mechanics for logits

Xavier Poncini

October 28, 2025

Computational mechanics is a framework that studies structures underlying optimal prediction. In recent times, it has emerged as a candidate to explain the internal representations of contemporary machine learning models. A naive view of computational mechanics applied to this setting suggests that for neural networks trained on a data generating process, the predictive vector of the process is decodable from the network activations. We apply this framework to study transformers trained to perform modular addition – a classic problem in mechanistic interpretability. While a hidden Markov model (HMM) exists for modular addition, we find that the transformer represents the predictive vectors of a new type of process that we refer to as the energy-based HMM (EHMM) – analogous to an HMM but produces logits instead of probabilities, that approximates the HMM process. In this report, we introduce the EHMM in a general setting and identify the corresponding notion of predictive vectors. We then focus on the problem of modular addition, where we present empirical evidence that the corresponding EHMM predictive vectors are linearly represented in the residual stream of a trained transformer.

Contents

1 *Introduction* 2
2 *(Just enough) Computational mechanics* 3
2.1 *Hidden Markov models* 4
2.2 *Energy-based hidden Markov models* 7
3 *Modular addition* 9
3.1 *Representations theory* 10
3.2 *Hidden Markov model* 11
3.3 *Energy-based hidden Markov model* 14
4 *Results* 17
5 *Discussion* 23
A *Generalised hidden Markov models* 24
B *Conditional logits* 25
C *Representation theory* 26
D *Transformer architecture details* 27

1 Introduction

Neural networks trained on the simple task of next-token prediction are endowed with incredible general-reasoning capabilities (Radford et al., 2019; Kaplan et al., 2020; Brown et al., 2020; Bubeck et al., 2023). This remarkable observation has fuelled industrial-scale investment into developing so-called *frontier large language models*. Investment continues despite our inability to understand the trillion-parameter networks that emerge during training. The field of *mechanistic interpretability* has grown to meet this challenge, with the ambitious goal of reverse engineering the computations performed by the weights of neural networks (Olah, 2023). Despite promising early progress (Elhage et al., 2022; Cunningham et al., 2023; Templeton et al., 2024; Lindsey et al., 2025), much work remains before the internal workings of frontier models are well-understood.

Computational mechanics is concerned with the computational properties of physical systems, specifically how they store and processes information (Crutchfield and Young, 1989; Crutchfield, 2011). A particularly relevant aspect of this framework is that it describes the structures required to achieve optimal prediction (Shalizi and Crutchfield, 1999). Given the emphasis on next-token prediction, some have suggested (Shai et al., 2024) that insights from computational mechanics may be useful for mechanistic interpretability. Recent studies have supported this insight; neural networks trained on data produced by linear generative processes[1] have been shown to e.g., linearly encode the *predictive vector*[2] of the underlying process (Shai et al., 2024; Pepper, 2024; Riechers et al., 2025b), and to satisfy in-context loss corresponding to the *myopic entropy rate*[3] (Riechers et al., 2025a).

[1] E.g, hidden Markov models (HMMs) and generalisations thereof.

[2] A quantity that corresponds to Bayesian updating over the hidden state of the process.

[3] An information-theoretic measure of the in-context uncertainty associated with a process.

While impressive, it is not clear whether insights from computational mechanics translate to neural networks that are:

1. Trained on processes that are in the training distribution of frontier models.

2. Industrial scale, i.e., have more than one billion parameters.

In this report, we address point 1, by considering toy-scale transformers trained to perform modular addition – a classic problem in mechanistic interpretability (Power et al., 2022; Nanda et al., 2023), with most mechanistic details now well understood (Yip et al., 2024; Wu et al., 2024). How, if at all, do these details interact with computational mechanics? We find that modular addition can be formulated in terms of a hidden Markov model (HMM) and that transformers trained on this process *do not* linearly represent the corresponding predictive vectors, contra (Shai et al., 2024; Riechers et al., 2025b).

Instead, we find that models linearly represent the predictive vector of a process *approximating* the HMM, and that this approximating process is an example of a new class of generative models that we call *energy-based hidden Markov model* (EHMMs). We believe EHMMs to be of independent interest. This class of generative models can be viewed as analogues of HMMs, but instead of assigning probabilities to sequences of tokens via a linear dynamic, they first assign *logits*, which are then passed through the *softmax function* to produce probabilities.

Finally, there is an early indication that our results may also address point 2 above. In (Kantamneni and Tegmark, 2025), the internal representations of industrial-scale transformers completing arithmetic tasks were shown to represent numbers as points on a *generalised helix*. While modular addition for large modulus was not studied directly, the helical structure identified inside large language models resembles the predictive vectors of the EHMM approximating modular addition. This suggests that the results reported here on toy models, may extend beyond this regime, and be relevant at industrial-scales.

We proceed by outlining the remaining sections of this report. In section 2, we recall the necessary computational mechanics preliminaries including HMMs and the associated predictive vector. We also define EHMMs and derive the corresponding notion of predictive vectors. In section 3, we consider the problem of modular addition and show that it admits a description in terms of an HMM that we call *random-random Mod$_p$* (RRMod$_p$)[4]. We also show that there exists an EHMM that approximates RRMod$_p$, that we refer to as *soft random-random Mod$_p$* (sRRMod$_p$). In section 4, we study a single-layer single-head transformer trained on data from RRMod$_p$. We provide evidence that the predictive vector of sRRMod$_p$ is linearly decodable from the activations of the transformer at various positions in the residual stream. Finally, in section 5 we summarise the main results, address limitations and offer avenues for future investigation.

[4] Where $p \in \mathbb{N}$, corresponds to the modulus of the modular sum i.e., the p in $c = a + b \bmod p$.

2 *(Just enough) Computational mechanics*

In this section, we begin by introducing aspects of computational mechanics relevant to the optimal prediction of data produced by a *hidden Markov model* (HMM) – a discrete-time stochastic process that assigns probabilities to sequences of tokens via a *linear dynamic*. In this setting, the central object of study is the *predictive vector* which corresponds to Bayesian updating over the hidden states of the generative model. We then define a new class of stochastic processes that we refer to as *energy-based hidden Markov models* (EHMMs), which are similar to an HMMs, but produce *logits*[5] via an identical linear

[5] Equivalently, negative-energies.

dynamic. For the EHMM, we derive an analogue of the predictive vector and show that it satisfies a simple recursive updating formula.

2.1 *Hidden Markov models*

A (Mealy-type) *hidden Markov model* (HMM) consists of the following data:

- A set $\mathcal{X}$ of *tokens* and a set $\mathcal{S}$ of *hidden states*.
- A collection of *transition matrices* $(T^{(x)})_{x\in\mathcal{X}}$ whose matrix elements are defined[6]

$$T^{(x)}_{ij} := \Pr(x, s_j \,|\, s_i) \tag{1}$$

  where $s_i, s_j \in \mathcal{S}$ and $i, j = 0, 1, \ldots, |\mathcal{S}| - 1$.
- An *initial vector* $\langle \eta^{(\emptyset)}|$ whose elements are defined[7]

$$\left(\langle \eta^{(\emptyset)}|\right)_i := \Pr(s_i). \tag{2}$$

Intuitively, the transition matrices define a natural dynamic on the hidden states of an HMM where, at each time-step, there is a transition from one hidden state to another[8] and an emission of a token. The particular hidden state transition and token emission are sampled from a probability distribution defined by the HMM, that depends only on the token, the source (previous) hidden state and the destination (current) hidden state.

Formally, the probability of observing a sequence $x_{1:\ell} = x_1 x_2 \cdots x_\ell \in \mathcal{X}^\ell$, factorises as:[9]

$$\Pr(x_{1:\ell}) = \sum_{t_0,\ldots,t_\ell=0}^{|\mathcal{S}|-1} \Pr(s_{t_0}) \Pr(x_1, s_{t_1} \,|\, s_{t_0}) \Pr(x_2, s_{t_2} \,|\, s_{t_1}) \cdots \Pr(x_\ell, s_{t_\ell} \,|\, s_{t_{\ell-1}}). \tag{3}$$

Conveniently, the above expression for $\Pr(x_{1:\ell})$ can be restated in terms of the HMM transition matrices and the initial vector as

$$\Pr(x_{1:\ell}) = \langle \eta^{(\emptyset)} | T^{(x_{1:\ell})} | 1 \rangle, \qquad T^{(x_{1:\ell})} := T^{(x_1)} T^{(x_2)} \cdots T^{(x_\ell)} \tag{4}$$

where $|1\rangle$ is the column vector (with appropriate dimension) whose elements are all ones. Accordingly, we say that the process assigns probabilities to tokens via a *linear dynamic*.

Suppose we are tasked with predicting tokens emitted from an HMM, how should we go about making *optimal* predictions? A natural strategy is to predict the current hidden state occupied by the HMM and use this information to anticipate future tokens. By implementing this strategy, we will derive a well-known object in the computational mechanics literature known as the *predictive vector*[10].

[6] Under this definition, the HMM emits the token x as it transitions from s_i to s_j.

[7] Throughout, we adopt *bra-ket* notation where *left-angled* brackets e.g., $\langle a|$, denote row vectors and *right-angled* brackets e.g., $|b\rangle$, denote column vectors. As we will be considering real valued vectors, we define $|x\rangle := (\langle x|)^\intercal$.

[8] Possibly the same hidden state.

[9] Due to the Markov condition on state transitions and token emissions.

[10] By referring to this object as the predictive vector, we adopt the terminology of Riechers et al. In the computational mechanics literature, this object is referred to as the mixed-state presentation (MSP), see Crutchfield et al. It has also been referred to as the *belief state*, see Shai et al.

Bayes' rule tells us that the probability of occupying the hidden state s_j after observing the token x is given by

$$\Pr(s_j \mid x) = \frac{\Pr(x \mid s_j)\Pr(s_j)}{\Pr(x)} = \frac{\Pr(x, s_j)}{\Pr(x)} = \frac{\sum_{i=1}^{|\mathcal{S}|}\Pr(s_i)\Pr(x, s_j \mid s_i)}{\Pr(x)}, \quad (5)$$

where the final equality follows from marginalising over the hidden state of the source. Similarly, Bayesian updating given the observation of the token sequence $x_{1:\ell}$ is given by

$$\Pr(s_j \mid x_{1:\ell}) = \quad (6)$$
$$\frac{\sum_{t_0,\ldots,t_{\ell-1}=0}^{|\mathcal{S}|-1}\Pr(s_{t_0})\Pr(x_1, s_{t_1} \mid s_{t_0})\Pr(x_2, s_{t_2} \mid s_{t_1})\cdots\Pr(x_\ell, s_\ell \mid s_{t_{\ell-1}})}{\Pr(x)}.$$

As above, $\Pr(s_j \mid x_{1:\ell})$ can be restated in terms of the HMM transition matrices and the initial vector as the following element

$$\Pr(s_j \mid x_{1:\ell}) = \left(\frac{\langle\eta^{(\varnothing)}|T^{(x_{1:\ell})}}{\langle\eta^{(\varnothing)}|T^{(x_{1:\ell})}|1\rangle}\right)_j. \quad (7)$$

The above expression motivates the definition of the *predictive vector* (Crutchfield et al., 2009) of the HMM

$$\langle\eta^{(x_{1:\ell})}| := \frac{\langle\eta^{(\varnothing)}|T^{(x_{1:\ell})}}{\langle\eta^{(\varnothing)}|T^{(x_{1:\ell})}|1\rangle}, \quad (8)$$

which corresponds to the probability distribution over hidden states of the HMM after a Bayesian update given the observed token sequence $x_{1:\ell}$ (Crutchfield, 2013). Suppose we observe a further token $x_{\ell+1}$, the updated predictive vector is a simple function of the previous predictive vector

$$\langle\eta^{(x_{1:\ell})}| \mapsto \langle\eta^{(x_{1:\ell+1})}| = \frac{\langle\eta^{(x_{1:\ell})}|T^{(x_{\ell+1})}}{\langle\eta^{(x_{1:\ell})}|T^{(x_{\ell+1})}|1\rangle}. \quad (9)$$

The predictive vector also plays a crucial role in conditional probabilities over future token emissions. Consider the probability over future token emissions $x_{1:\ell'}^{(f)}$ given the observation of the previous token emissions $x_{1:\ell}^{(p)}$

$$\Pr(x_{1:\ell'}^{(f)} \mid x_{1:\ell}^{(p)}) = \frac{\Pr(x_{1:\ell}^{(p)}, x_{1:\ell'}^{(f)})}{\Pr(x_{1:\ell}^{(p)})} = \frac{\langle\eta^{(\varnothing)}|T^{(x_{1:\ell}^{(p)})}T^{(x_{1:\ell'}^{(f)})}|1\rangle}{\langle\eta^{(\varnothing)}|T^{(x_{1:\ell}^{(p)})}|1\rangle}. \quad (10)$$

Expressing $\Pr(x_{1:\ell'}^{(f)} \mid x_{1:\ell}^{(p)})$ in terms of the predictive vector $\langle\eta^{(x_{1:\ell}^{(p)})}|$, we have

$$\Pr(x_{1:\ell'}^{(f)} \mid x_{1:\ell}^{(p)}) = \langle\eta^{(x_{1:\ell}^{(p)})}|T^{(x_{1:\ell'}^{(f)})}|1\rangle, \quad (11)$$

and observe that the predictive vector $\langle \eta^{(x^{(p)}_{1:\ell})}|$ is relevant to *all* future token predictions.

By considering strategies to perform optimal predictions of token emissions from an HMM, we have derived the predictive vector and have established some well-known facts about this object:

1. The predictive vector corresponds to Bayesian updating over the hidden states of the HMM, conditioned on a sequence of token emissions, see equation (7).
2. There is a simple recursive formula for updating the predictive vector, given further token observations, see equation (9).
3. All conditional probabilities can be expressed in terms of the appropriate predictive vector, see equation (11).

Notably, the relevance of the predictive vector transcends implementation details[11] of the predictor. Specialising these insights to neural networks trained to achieve near optimal performance on next-token prediction, motivates the following hypothesis:

[11] Details worth considering are substrate (e.g., biological or artificial) and architectural (i.e., the mechanism of the predictor).

Comp-mech hypothesis. For neural networks trained[12] on HMM data, the predictive vector is decodable from network activations.

[12] To near-optimal performance via stochastic gradient descent (SGD) or similar.

Evidence for the comp-mech hypothesis first appeared in (Shai et al., 2024), where it was shown that the predictive vector is *linearly* decodable from the activations of a transformer trained on two distinct HMM processes[13]. Subsequent work demonstrated a similar result across neural network architectures including recurrent neural networks (RNNs) (Pepper, 2024), long short-term memory units (LSTMs) and gated recurrent units (GRUs) (Riechers et al., 2025b), prompting a strengthening of the comp-mech hypothesis:

[13] Mess3 and random-random XOR (RRXOR).

Strong comp-mech hypothesis. For neural networks trained on HMM data, the predictive vector is *linearly* decodable from network activations.

The observation that the predictive vector is linearly represented in the activations of neural networks trained to near-optimal performance suggests deep connections to the *linear representation hypothesis*, see e.g. (Park et al., 2023). The importance of training dynamics in giving rise to linear representations is highlighted in (Jiang et al., 2024), where it is suggested that the learning process introduces an inductive bias towards linear representations. These findings are supported by the case study (Goldstein, 2024), where it was shown that a transformer, with hand-crafted weights, can perform optimal prediction for the *random-random XOR* (RRXOR) process and *does not* admit a linear representation of the predictive vector of this process[14].

[14] It remains to identify whether the predictive vector is, at all, decodable from the activations of this model.

In sections 3 and 4, we probe the applicability of the comp-mech hypotheses and find that the strong form does not directly apply to the familiar problem of modular addition. We remedy this situation by (1) introducing a new class of data generating processes, that operates at the level of the logits, and that admits an example approximating the modular addition process; and (2) suggesting a relaxation of the strong comp-mech hypothesis that incorporates this new class of processes and, under which, modular addition applies. We proceed by defining this new class.

2.2 *Energy-based hidden Markov models*

It is common practice in contemporary neural network architectures[15] to produce a probability distribution over a set S of possible outputs by passing the real-valued vector of outputs of the final layer $(z(x))_{x\in S}$ through the *softmax function*[16]

$$\Pr(x) = \frac{e^{z(x)}}{\sum_{x\in S} e^{z(x)}}. \tag{12}$$

[15] E.g. Multi-layer perceptrons (MLPs), recurrent neural networks (RNNs), transformers and convolutional neural networks (CNNs).

[16] Here we have fixed the temperature to one for simplicity.

The inputs to the softmax function $(z(x))_{x\in S}$, are referred to as the *logits* of the model. It is convenient to note that the softmax function is invariant under a constant shift of the logits, i.e., $(z(x)+c)_{x\in S}$ and $(z(x))_{x\in S}$ give rise to the same probability distribution over S:

$$\Pr_c(x) = \frac{e^{z(x)+c}}{\sum_{x\in S} e^{z(x)+c}} = \frac{e^c e^{z(x)}}{e^c \sum_{x\in S} e^{z(x)}} = \frac{e^{z(x)}}{\sum_{x\in S} e^{z(x)}} = \Pr(x) \tag{13}$$

where $c \in \mathbb{R}$.

The softmax function is well-known in the physics literature and goes by the name of the *Boltzmann distribution*. In this setting, the logits take the form

$$z(x) = -E_x\beta \tag{14}$$

where E_x is the *energy* of the state $x \in S$, and β is the *inverse temperature* of the system. The Boltzmann distribution describes the probability of a system, in thermal equilibrium with inverse temperature β, occupying the state x. Reinterpreting the shift-invariance property (13) in this context, we see that the physics of this system[17] depends only on the differences in energies between states.

[17] I.e., the distribution of states.

An *energy-based hidden Markov model* (EHMM) consists of the following data:

- A set $\mathcal{X}$ of *tokens*.
- A collection of *transition matrices* $(H^{(x)})_{x\in\mathcal{X}}$.
- An *initial vector* $\langle\eta^{(\emptyset)}|$ and a *final vector* $|\varphi\rangle$.

such that

$$\langle \eta^{(\emptyset)} | H^{(x_{1:\ell})} | \varphi \rangle \in \mathbb{R}, \qquad H^{(x_{1:\ell})} := H^{(x_1)} H^{(x_2)} \cdots H^{(x_\ell)} \tag{15}$$

for all $x_{1:\ell} \in \mathcal{X}^\ell$ and for all $\ell \in \mathbb{N}$. Accordingly, an EHMM defines a discrete-time stochastic process by first assigning logits[18] to each sequence of tokens of length ℓ via a linear dynamic on transition matrices

[18] Equivalently, negative energies.

$$z(x_{1:\ell}) = \langle \eta^{(\emptyset)} | H^{(x_{1:\ell})} | \varphi \rangle, \tag{16}$$

then probabilities via the softmax function (12).

To appreciate the resemblance between EHMMs and HMMs, note that HMMs produce probabilities directly via a linear dynamic analogous to (16), see equation (4). Unlike HMMs, EHMMs do not make direct reference to hidden states of the underlying stochastic process, in this way, they are closer to generalised hidden Markov models (GHMMs). GHMMs define probabilities via a linear dynamic as in (4), but relax the stochasticity condition on the transition matrices and initial vector, see appendix A for further details.

We proceed by defining an analogue of the predictive vector for EHMMs, such that the relevant properties outlined in the paragraph following equation (11), still hold for the EHMM predictive vector. In this setting, our defining relation will be an analogue of property 3, but for logits.

We refer to *conditional logits* as the logits $z(y \mid x)$, that after softmax, produce the conditional probabilities

$$\Pr(y \mid x) = \frac{e^{z(y \mid x)}}{\sum_{y \in S} e^{z(y \mid x)}}. \tag{17}$$

where x and y are realisations of some, possibly distinct, random variable. Simple manipulations, see Proposition B.1 in appendix B for details, show that the conditional logits, can be expressed simply in terms of the joint logits

$$z(y \mid x) = z(x, y). \tag{18}$$

Now, consider the conditional logits over future token emissions $x^{(f)}_{1:\ell'}$ given the observation of the previous token emissions $x^{(p)}_{1:\ell}$

$$z(x^{(f)}_{1:\ell'} \mid x^{(p)}_{1:\ell}) = z(x^{(p)}_{1:\ell} x^{(f)}_{1:\ell'}) = \langle \eta^{(\emptyset)} | H^{(x^{(p)}_{1:\ell})} H^{(x^{(f)}_{1:\ell'})} | \varphi \rangle, \tag{19}$$

we define the predictive vector of the EHMM as the vector that is relevant to *all* future token predictions[19]

[19] Via logits.

$$\langle \eta^{(x_{1:\ell})} | := \langle \eta^{(\emptyset)} | H^{(x_{1:\ell})}. \tag{20}$$

Expressing the conditional logits in terms of the predictive vector we have

$$z(x^{(f)}_{1:\ell'} \mid x^{(p)}_{1:\ell}) = \langle \eta^{(x^{(p)}_{1:\ell})} | H^{(x^{(f)}_{1:\ell'})} | 1 \rangle \tag{21}$$

which is a direct analogue of equation (11), but in the EHMM setting. Suppose we observe a further token $x_{\ell+1}$, the updated predictive vector is a simple function of the previous predictive vector

$$\langle \eta^{(x_{1:\ell})} | \mapsto \langle \eta^{(x_{1:\ell+1})} | = \langle \eta^{(x_{1:\ell})} | H^{(x_{\ell+1})}. \tag{22}$$

A distinctive feature of the EHMM predictive vector is that the recursion (22) is a linear transformation of the previous vector, whereas in the HMM setting, it is given by a linear transformation *and* a scaling (9). A simple consequence of this fact is that there exists no EHMM process that exhibits a fractal dynamic on the predictive vector, this is contrasted with the HMM case which is known to give rise to fractal structure e.g., the Mess3 process (Marzen and Crutchfield, 2017).

In table 1, we summarise the key features of the classes of generative processes introduced in sections 2.1 and 2.2. In the following section, we apply this formalism to the classic problem of modular addition. In this setting, we find that there exists an HMM process that exactly describes this problem, and an EHMM process that approximates this problem.

Process	Output	Predictive vector	Recursion
HMM	$\Pr(x_{1:\ell}) = \langle \eta^{(\varnothing)} \vert T^{(x_{1:\ell})} \vert 1 \rangle$	$\langle \eta^{(x_{1:\ell})} \vert = \frac{\langle \eta^{(\varnothing)} \vert T^{(x_{1:\ell})}}{\langle \eta^{(\varnothing)} \vert T^{(x_{1:\ell})} \vert 1 \rangle}$	$\langle \eta^{(x_{1:\ell+1})} \vert = \frac{\langle \eta^{(x_{1:\ell})} \vert T^{(x_{\ell+1})}}{\langle \eta^{(x_{1:\ell})} \vert T^{(x_{\ell+1})} \vert 1 \rangle}$
EHMM	$z(x_{1:\ell}) = \langle \eta^{(\varnothing)} \vert H^{(x_{1:\ell})} \vert \varphi \rangle$	$\langle \eta^{(x_{1:\ell})} \vert = \langle \eta^{(\varnothing)} \vert H^{(x_{1:\ell})}$	$\langle \eta^{(x_{1:\ell+1})} \vert = \langle \eta^{(x_{1:\ell})} \vert H^{(x_{\ell+1})}$

Table 1: A summary of aspects relevant to the definition and prediction of hidden Markov models (HMM) and energy-based hidden Markov models (EHMM).

3 *Modular addition*

The problem of modular addition admits a natural formulation as next-token prediction by training a neural network on sequences from the set[20,21]

$$D_p := \{ abc \mid c = a + b \bmod p,\ a, b \in \{0, 1, \ldots, p-1\} \}, \tag{23}$$

where $p \in \mathbb{N}$. Sampling $x_{1:3}$ uniformly from the set D_p, we have the following probabilities:

$$\begin{aligned} \Pr(x_{1:1} = a) = \frac{1}{p}, \qquad \Pr(x_{1:2} = ab) = \frac{1}{p^2}, \\ \Pr(x_{1:3} = abc) = \frac{\delta_{c,\, a+b \bmod p}}{p^2}. \end{aligned} \tag{24}$$

[20] This approach can be applied to algebraic structures more generally, e.g. groups & monoids.

[21] For brevity, we will often omit commas between elements in a sequence, i.e., we will write a, b, c simply as abc.

To make use of the computational mechanics formalism introduced in section 2, we will define processes (an HMM and an EHMM) that assign probabilities as in (24). To this end, we find it convenient to view modular addition as the composition of elements of the *cyclic group*[22] and utilise some basic results from *representation theory*.

[22] Taking inspiration from Chughtai et al.

3.1 *Representations theory*

Here, we recall some basic facts about the representation theory of the cylic group. We assume familiarity with basic notions of representation theory[23], see appendix C for a refresher.

[23] Including groups, representations and group actions.

For each $p \in \mathbb{N}$, the *cyclic group* C_p is generated by a single element r subject to

$$r^p = \mathrm{id}, \tag{25}$$

where $r^0 := \mathrm{id}$ is the identity element. The canonical set of group elements is given by $\{\mathrm{id}, r, r^2, \ldots, r^{p-1}\}$, and the relation to modular addition can be stated as

$$r^c = r^{a+b} \quad \Longleftrightarrow \quad c = a + b \bmod p. \tag{26}$$

We now consider two distinct C_p-actions.

$(p-1)$**-simplex.** Define the following set

$$S_p := \{\langle e_p^{(i)} | \,|\, i = 0, 1, \ldots, p-1\}, \qquad (\langle e_p^{(i)}|)_j := \delta_{i,j} \tag{27}$$

where $\langle e_p^{(i)}|$ denotes the p-dimensional *elementary row-vector*. Consider the C_p-action

$$\sigma : S_p \times C_p \to S_p, \qquad (\langle e_p^{(a)}|, r^b) \mapsto \langle e_p^{(a+b \bmod p)}|, \tag{28}$$

which induces the representation[24]

[24] We use $\mathrm{Mat}_{m\times n}(S)$ to denote the set of $m \times n$ matrices whose matrix elements are in the set S.

$$\rho_\sigma : C_p \to \mathrm{Mat}_{p\times p}(\{0,1\}), \qquad r \mapsto \begin{bmatrix} 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \ddots & \vdots \\ \vdots & 0 & 0 & \ddots & 0 \\ 0 & \vdots & \ddots & \ddots & 1 \\ 1 & 0 & \cdots & 0 & 0 \end{bmatrix}. \tag{29}$$

Geometrically, each element of S_p corresponds to a vertex of the $(p-1)$-simplex, and the corresponding representation acts as a cyclic permutation among the vertices.

p**-gon.** For each $\omega = 1, \ldots, \lfloor \frac{p}{2} \rfloor$, define the following set

$$V_p^{(\omega)} := \{\langle v_\omega^{(i)} | \,|\, i = 0, 1, \ldots, p-1\}, \tag{30}$$

where

$$\langle v_\omega^{(i)}| := \begin{bmatrix} \cos(\frac{2\pi\omega i}{p}) & \sin(\frac{2\pi\omega i}{p}) \end{bmatrix}. \tag{31}$$

Consider the C_p-action

$$\gamma^{(\omega)} : V_p^{(\omega)} \times C_p \to V_p^{(\omega)}, \qquad (\langle v_\omega^{(a)}|, r^b) \mapsto \langle v_\omega^{(a+b)}| \tag{32}$$

which induces the representation

$$\rho_{\gamma^{(\omega)}} : C_p \to \mathrm{Mat}_{2\times 2}(\mathbb{R}), \qquad r \mapsto \begin{bmatrix} \cos(\frac{2\pi\omega}{p}) & \sin(\frac{2\pi\omega}{p}) \\ -\sin(\frac{2\pi\omega}{p}) & \cos(\frac{2\pi\omega}{p}) \end{bmatrix}. \tag{33}$$

Geometrically, each element of $V_p^{(\omega)}$ corresponds to a vertex of a p-gon[25] in the plane, and the corresponding representation acts as a two-dimensional rotation by the angle $\frac{2\pi\omega}{p}$.

[25] p-sided polygon.

3.2 *Hidden Markov model*

Define the vocabulary

$$\mathcal{X}_p := \{0, 1, \ldots, p-1\}, \tag{34}$$

the set of hidden states

$$\mathcal{S}_p := \{s^{(0)}\} \cup \{s_0^{(1)}, s_1^{(1)}, \ldots s_{p-1}^{(1)}\} \cup \{s_0^{(2)}, s_1^{(2)}, \ldots s_{p-1}^{(2)}\}, \tag{35}$$

and the initial vector

$$\langle \eta^{(\emptyset)}| := \begin{bmatrix} 1 & \mathbf{0}_{1\times p} & \mathbf{0}_{1\times p} \end{bmatrix}, \tag{36}$$

where $\mathbf{0}_{m\times n}$ denotes an $m \times n$ block of zeros. With respect to the natural ordering of the hidden states as in (35), we define the transition matrices as

$$T_p^{(x)} := \begin{bmatrix} 0 & \frac{1}{p}\langle e_p^{(x)}| & \mathbf{0}_{1\times p} \\ \mathbf{0}_{p\times 1} & \mathbf{0}_{p\times p} & \frac{1}{p}\rho_\sigma(r^x) \\ |e_p^{(x)}\rangle & \mathbf{0}_{p\times p} & \mathbf{0}_{p\times p} \end{bmatrix}. \tag{37}$$

where $\langle e_p^{(x)}|$ and ρ_σ are defined in (27) and (29) respectively.

The tuple $(\mathcal{X}_p, \mathcal{S}_p, (T_p^{(x)})_{x\in\mathcal{X}_p}, \langle \eta^{(\emptyset)}|)$ is an HMM that we refer to as the *random-random Mod*$_p$ (RRMod$_p$) process. We proceed by showing that this process gives rise to the probabilities (24):

$$\Pr(x_{1:1} = a) = \langle \eta^{(\emptyset)}|T_p^{(a)}|1\rangle = \begin{bmatrix} 0 & \frac{1}{p}\langle e_p^{(a)}| & \mathbf{0}_{1\times p} \end{bmatrix} \begin{bmatrix} 1 \\ \mathbf{1}_{p\times 1} \\ \mathbf{1}_{p\times 1} \end{bmatrix} = \frac{1}{p}, \tag{38}$$

$$\begin{aligned}\Pr(x_{1:2} = ab) = \langle \eta^{(\varnothing)} | T_p^{(a)} T_p^{(b)} | 1 \rangle &= \begin{bmatrix} 0 & \mathbf{0}_{1\times p} & \frac{1}{p^2} \langle e_p^{(a)} | \rho_\sigma(r^b) \end{bmatrix} \begin{bmatrix} 1 \\ \mathbf{1}_{p\times 1} \\ \mathbf{1}_{p\times 1} \end{bmatrix} \\ &= \begin{bmatrix} 0 & \mathbf{0}_{1\times p} & \frac{1}{p^2} \langle e_p^{(a+b \bmod p)} | \end{bmatrix} \begin{bmatrix} 1 \\ \mathbf{1}_{p\times 1} \\ \mathbf{1}_{p\times 1} \end{bmatrix} = \frac{1}{p^2},\end{aligned} \tag{39}$$

$$\begin{aligned}\Pr(x_{1:3} = abc) &= \langle \eta^{(\varnothing)} | T_p^{(a)} T_p^{(b)} T_p^{(c)} | 1 \rangle \\ &= \begin{bmatrix} \frac{1}{p^2} \langle e_p^{(a+b \bmod p)} | e_p^{(c)} \rangle & \mathbf{0}_{1\times p} & \mathbf{0}_{1\times p} \end{bmatrix} \begin{bmatrix} 1 \\ \mathbf{1}_{p\times 1} \\ \mathbf{1}_{p\times 1} \end{bmatrix} = \frac{\delta_{c,\, a+b \bmod p}}{p^2}\end{aligned} \tag{40}$$

where $x_{1:3} = abc \in \mathcal{X}_p^3$ and $\mathbf{1}_{m\times n}$ denotes an $m \times n$ block of ones. To provide some further intuition, we describe the process to produce the output string abc:

0. *Initialisation*. The process is initialised in the state $s^{(0)}$. This corresponds to the vector $\langle \eta^{(\varnothing)} |$.

1. *First random*. An element a is sampled uniformly from $\mathcal{X}_p$, the state transitions from $s^{(0)}$ to $s_a^{(1)}$ and emits the token a. This corresponds to drawing the vector $\langle e_p^{(a)} |$.

2. *Second random*. An element b is sampled uniformly from $\mathcal{X}_p$, and the state transitions from $s_a^{(1)}$ to $s_{a+b \bmod p}^{(2)}$ and emits the token b. This corresponds to drawing the matrix $\rho_\sigma(r^b)$ and computing the action $\langle e_p^{(a)} | \rho_\sigma(r^b) = \langle e_p^{(a+b \bmod p)} |$.

3. *Final transition*. The state transitions from $s_{a+b \bmod p}^{(2)}$ to $s^{(0)}$ and emits the token $c = a + b \bmod p$. This corresponds to computing the inner product $\langle e_p^{(a+b \bmod p)} | e_p^{(c)} \rangle = \delta_{c, a+b \bmod p}$.

The predictive vectors of the RRMod_p process are given by

$$\langle \eta^{(\varnothing)} | := \begin{bmatrix} 1 & \mathbf{0}_{1\times p} & \mathbf{0}_{1\times p} \end{bmatrix} \tag{41}$$

$$\langle \eta^{(a)} | := \frac{\langle \eta^{(\varnothing)} | T_p^{(a)}}{\langle \eta^{(\varnothing)} | T_p^{(a)} | 1 \rangle} = \begin{bmatrix} 0 & \langle e_p^{(a)} | & \mathbf{0}_{1\times p} \end{bmatrix} \tag{42}$$

$$\langle \eta^{(ab)} | := \frac{\langle \eta^{(\varnothing)} | T_p^{(a)} T_p^{(b)}}{\langle \eta^{(\varnothing)} | T_p^{(a)} T_p^{(b)} | 1 \rangle} = \begin{bmatrix} 0 & \mathbf{0}_{1\times p} & \langle e_p^{(a+b \bmod p)} | \end{bmatrix} \tag{43}$$

$$\langle \eta^{(abc)} | := \frac{\langle \eta^{(\varnothing)} | T_p^{(a)} T_p^{(b)} T_p^{(c)}}{\langle \eta^{(\varnothing)} | T_p^{(a)} T_p^{(b)} T_p^{(c)} | 1 \rangle} = \begin{cases} \langle \eta^{(\varnothing)} |, & c = a + b \bmod p \\ \text{undefined}, & \textit{else} \end{cases} \tag{44}$$

where we have reused components of the calculations in (38)–(40). The predictive vectors at initialisation (41) and after observing the tokens *abc* (44), are identical – corresponding to a single point in a one-dimensional subspace. The non-trivial predictive vectors occur after observing the sequence *a* (42) and *ab* (43). These vectors live in two distinct p-dimensional subspaces, and both take values in the set S_p. Recalling the geometric interpretation of S_p elements, the predictive vectors of RRMod_p lie on the vertices of a $(p-1)$-simplex. In the language of (Riechers et al., 2025b), these correspond to *pure states* of the generative process.

While, to the best of our knowledge, RRMod_p has not appeared in the literature[26], it is a natural generalisation of the random-random XOR (RRXOR) process (Crutchfield and Feldman, 2001) – a well-studied process in the computational mechanics literature, see (Still et al., 2007; Riechers and Crutchfield, 2017) for example. For $p = 2$

$$\mathrm{RRMod}_p = \mathrm{RRXOR}, \tag{45}$$

which follows directly from the fact that addition modulo two is equivalent to the XOR operation on bits. Typically, RRXOR is studied when initialised in the stationary distribution of the marginalised transition matrix, i.e. in the state $\langle \pi |$ satisfying $\langle \pi | T = \langle \pi |$ where $T := \sum_{x \in \mathcal{X}_2} T^{(x)}$. In this setting, the predictive vector[27] of this process has been shown to be linearly decodable from the activations of a trained transformer (Shai et al., 2024).

[26] It is likely known to experts.

[27] Note that when initialised in the stationary distribution, (41)–(44) are a *subset* of the full set of predictive vectors for this process.

3.3 *Energy-based hidden Markov model*

Fix $n \in \mathbb{N}$, $\alpha_k \in \mathbb{R}_{>0}$ and $\omega_k \in \{1, \dots, \lfloor \frac{p}{2} \rfloor\}$ for each $k = 1, \dots, n$, and introduce the following auxiliary objects

$$\langle v^{(i)}| := \bigoplus_{k=1}^{n} \alpha_k \langle v^{(i)}_{\omega_k}|, \qquad \rho_\gamma(r) := \bigoplus_{i=1}^{n} \rho_{\gamma^{(\omega_k)}}(r). \tag{46}$$

where $\langle v^{(i)}_{\omega_k}|$ and $\rho_{\gamma^{(\omega_k)}}$ are defined in (31) and (33) respectively. Define the vocabulary[28]

$$\mathcal{X}_p := \{0, 1, \dots, p-1\}, \tag{47}$$

the transition matrices

$$H_p^{(x)} := \begin{bmatrix} 0 & \frac{1}{p}\langle v^{(x)}| & \mathbf{0}_{1\times 2n} \\ \mathbf{0}_{2n\times 1} & \mathbf{0}_{2n\times 2n} & \frac{1}{p}\rho_\gamma(r^x) \\ |v^{(x)}\rangle & \mathbf{0}_{2n\times 2n} & \mathbf{0}_{2n\times 2n} \end{bmatrix}, \tag{48}$$

and the initial[29] and final[30] vectors

$$\langle \eta^{(\emptyset)}| := \begin{bmatrix} 1 & \mathbf{0}_{1\times 2n} & \mathbf{0}_{1\times 2n} \end{bmatrix}, \qquad |\varphi\rangle = \begin{bmatrix} 1 \\ \mathbf{0}_{2n\times 1} \\ \mathbf{0}_{2n\times 1} \end{bmatrix}. \tag{49}$$

The tuple $(\mathcal{X}_p, (H_p^{(x)})_{x\in\mathcal{X}_p}, \langle \eta^{(\emptyset)}|, |\varphi\rangle)$ is an EHMM that we refer to as the *soft random-random Mod$_p$* (sRRMod$_p$) process. We proceed by showing that this process *approximates* the probabilities (24), by first computing the logits:

$$z(x_{1:1} = a) = \langle \eta^{(\emptyset)}|H_p^{(a)}|\varphi\rangle = \begin{bmatrix} 0 & \frac{1}{p}\langle v^{(a)}| & \mathbf{0}_{1\times 2n} \end{bmatrix} \begin{bmatrix} 1 \\ \mathbf{1}_{2n\times 1} \\ \mathbf{1}_{2n\times 1} \end{bmatrix} = 0, \tag{50}$$

$$\begin{aligned} z(x_{1:2} = ab) = \langle \eta^{(\emptyset)}|H_p^{(a)}H_p^{(b)}|\varphi\rangle &= \begin{bmatrix} 0 & \mathbf{0}_{1\times 2n} & \frac{1}{p^2}\langle v^{(a)}|\rho_\gamma(r^b) \end{bmatrix} \begin{bmatrix} 1 \\ \mathbf{1}_{2n\times 1} \\ \mathbf{1}_{2n\times 1} \end{bmatrix} \\ &= \begin{bmatrix} 0 & \mathbf{0}_{1\times 2n} & \frac{1}{p^2}\langle v^{(a+b)}| \end{bmatrix} \begin{bmatrix} 1 \\ \mathbf{1}_{2n\times 1} \\ \mathbf{1}_{2n\times 1} \end{bmatrix} = 0, \end{aligned} \tag{51}$$

$$\begin{aligned} z(x_{1:3} = abc) &= \langle \eta^{(\emptyset)}|H_p^{(a)}H_p^{(b)}H_p^{(c)}|\varphi\rangle \\ &= \begin{bmatrix} \frac{1}{p^2}\langle v^{(a+b)}|v^{(c)}\rangle & \mathbf{0}_{1\times 2n} & \mathbf{0}_{1\times 2n} \end{bmatrix} \begin{bmatrix} 1 \\ \mathbf{1}_{2n\times 1} \\ \mathbf{1}_{2n\times 1} \end{bmatrix} = \sum_{k=1}^{n} \frac{\alpha_k^2}{p^2} \cos\left(\frac{2\pi\omega_k(a+b-c)}{p}\right) \end{aligned} \tag{52}$$

[28] This is the same vocabulary as in RRMod$_p$.

[29] This is the same initial vector as in the RRMod$_p$ process defined in section 3.2.

[30] Indeed, we can express the final vector simply as the transpose of the initial vector. As this is a non-generic feature of EHMM processes, we opt to distinguish them as in (49).

where $x_{1:3} = abc \in \mathcal{X}_p^3$, and we emphasise that the functional form of the logits (52) matches those that have been shown to be empirically relevant (Nanda et al., 2023; Chughtai et al., 2023). Then, passing these logits through the softmax function

$$\Pr(x_{1:1} = a) = \frac{1}{p}, \qquad \Pr(x_{1:2} = ab) = \frac{1}{p^2},$$
$$\Pr(x_{1:3} = abc) = \frac{\exp\left(\frac{1}{p^2}\sum_{k=1}^{n}\alpha_k^2\cos\left(\frac{2\pi\omega_k(a+b-c)}{p}\right)\right)}{\sum_{def\in\mathcal{X}_p^3}\exp\left(\frac{1}{p^2}\sum_{k=1}^{n}\alpha_k^2\cos\left(\frac{2\pi\omega_k(d+e-f)}{p}\right)\right)}. \tag{53}$$

Comparing (24) and (53), we see that the probabilities of observing the token sequences a and ab for sRRMod_p correspond exactly to the modular addition probabilities[31], however, the probability assigned to the sequence abc only *approximates* the ground truth distribution.

[31] I.e., the uniform distribution of tokens.

The ground truth distribution concentrates the probability of outputting the string abc on the correct modular sum i.e., c satisfying $c = a + b \bmod p$, and assigns zero probability to all other outputs. To contrast, sRRMod_p associates a non-zero probability to all strings $abc \in \mathcal{X}_p^3$, even incorrect answers! Crucially, the string corresponding to the correct modular sum is assigned the largest probability[32], see (53). Moreover, the free parameters n and $(\alpha_k, \omega_k)_{k=1,\ldots,n}$ of sRRMod_p can be tuned to better approximate the ground truth distribution (24). If n is increased such that there are more distinct frequencies $(\omega_k)_{k=1,\ldots,n}$, each of the terms in the sum constructively interfere on the correct token sequence and destructively interfere on incorrect token sequence (Chughtai et al., 2023), which contribute to increasing the probability of the correct sequence. Alternatively, fix n and increase each of the coefficients $(\alpha_k)_{k=1,\ldots,n}$, this increases the relative distance between the correct and incorrect token sequences, which similarly contributes to a better approximation of the probabilities (24).

[32] Which is a direct consequence of being assigned the largest logit.

Here, we describe the process to produce the output string abc:

0. *Initialisation*. The process is initialised in the vector $\langle\eta^{(\varnothing)}|$.

1. *First random*. An element a is sampled uniformly from $\mathcal{X}_p$, which corresponds to transitioning to the vector $\langle v^{(a)}|$ and emitting the token a.

2. *Second random*. An element b is sampled uniformly from $\mathcal{X}_p$, which corresponds to drawing the matrix $\rho_\gamma(r^b)$, transitioning to the vector $\langle v^{(a)}|\rho_\gamma(r^b) = \langle v^{(a+b \bmod p)}|$ and emitting the token b.

3. *Final transition*. The process transitions to the initial vector $\langle\eta^{(\varnothing)}|$ and emits the token c with probability determined by the logit value $\langle v^{(a+b \bmod p)}|v^{(c)}\rangle$.

A key distinction between the process described above and that described in section 3.2 is absence of explicit hidden states underlying the process. Despite this, sRRMod$_p$ maintains the same structure as RRMod$_p$, by producing a stochastic process via a C_p-action.

The predictive vectors of the sRRMod$_p$ process are given by

$$\langle \eta^{(\emptyset)}| := \begin{bmatrix} 1 & \mathbf{0}_{1\times 2n} & \mathbf{0}_{1\times 2n} \end{bmatrix} \tag{54}$$

$$\langle \eta^{(a)}| := \langle \eta^{(\emptyset)}|H_p^{(a)} = \begin{bmatrix} 0 & \frac{1}{p}\langle v^{(a)}| & \mathbf{0}_{1\times 2n} \end{bmatrix} \tag{55}$$

$$\langle \eta^{(ab)}| := \langle \eta^{(\emptyset)}|H_p^{(a)}H_p^{(b)} = \begin{bmatrix} 0 & \mathbf{0}_{1\times 2n} & \frac{1}{p^2}\langle v^{(a+b \bmod p)}| \end{bmatrix} \tag{56}$$

$$\langle \eta^{(abc)}| := \langle \eta^{(\emptyset)}|H_p^{(a)}H_p^{(b)}H_p^{(c)} = \sum_{k=1}^{n} \frac{\alpha_k^2}{p^2} \cos\left(\frac{2\pi\omega_k(a+b-c)}{p}\right) \langle \eta^{(\emptyset)}| \tag{57}$$

where we have reused components of the calculations in (50)–(52). The predictive vectors at initialisation (54) and after observing the tokens *abc* (57), are identical – corresponding to a single point in a one-dimensional subspace. As with RRMod$_p$, the non-trivial predictive vectors occur after observing the sequence *a* (55) and *ab* (56). These vectors live in two distinct $2n$-dimensional subspaces, and both take values in the set

$$V_p := \left\{ \langle v^{(i)}| \,\middle|\, i = 0, 1, \ldots, p-1 \right\}. \tag{58}$$

Recalling the geometric interpretation of $V_p^{(\omega)}$ elements[33], the predictive vectors of sRRMod$_p$ lie on the vertices of n distinct p-gon.

[33] See the paragraph including equation (33).

Process	Modular addition	Predictive vectors
RRMod$_p$	Exact	$(p-1)$-simplex: $S_p = \left\{ \langle e_p^{(i)}| \,\middle|\, i = 0, 1, \ldots, p-1 \right\}$
sRRMod$_p$	Approximation	p-gon: $V_p = \left\{ \langle v^{(i)}| \,\middle|\, i = 0, 1, \ldots, p-1 \right\}$

Table 2: A summary of properties of the generative processes RRMod$_p$ and sRRMod$_p$, related to the problem of modular addition.

In table 2, we summarise the key features of the generative processes modelling modular addition, introduced in sections 3.2 and 3.3. An orthodox view of the comp-mech hypothesis introduced in section 2.1, suggests that a neural network trained on data from RRMod$_p$ will encode structures, in particular the predictive vector, of this process. This view is in tension with the observation that the logits of neural networks trained on data from RRMod$_p$ take the functional form of the sRRMod$_p$ process (52). In the following section, we resolve this tension by identifying the form of the predictive vectors linearly represented in the activations of a transformer trained on RRMod$_p$.

4 Results

We adopt the general structure of the (Power et al., 2022) setup for training toy-scale transformers on modular addition tasks. For fixed $p \in \mathbb{N}$, we train a transformer on sequences from the set (23) augmented by a beginning of sequence token (BOS) i.e., of the form BOSabc where $c = a + b \bmod p$. Our sequences omit the composition ('+') and equality ('=') tokens which can be inferred from sequence position. Next-token prediction loss is only computed with respect to the final token position i.e., c for the sequence abc.

For simplicity, we consider small transformer architectures and find that a single-layer single-head transformer is capable of *grokking*[34] modular addition for p-values ranging from 73 to 113. To distinguish the process[35] relevant to the internal representations of the transformer, we implement the following procedure:

[34] Achieving high accuracy on a validation set.

[35] RRMod_p or sRRMod_p.

1. Fix $p \in \mathbb{N}$ and train a transformer such that it groks modular addition.

2. Identify the free-parameters, $(\alpha_k)_{k=1,\dots,n}$ and $(\omega_k)_{k=1,\dots,n}$, of sRRMod_p, by fitting the logits of the trained transformer to the functional form (Nanda et al., 2023):

$$\sum_{k=1}^{n} \alpha_k^2 \cos\left(\frac{2\pi\omega_k(a+b-c)}{p}\right). \tag{59}$$

3. Analyse the model activations for consistency with the predictive vectors of RRMod_p and sRRMod_p.

For the remainder of this section, we fix $p = 113$ and present results for a single-layer single-head transformer with a 128-dimensional[36] residual stream, see appendix D for the full architectural details. It has been argued (Chughtai et al., 2023) that the parameters $(\alpha_k)_{k=1,\dots,n}$ and $(\omega_k)_{k=1,\dots,n}$ learned by neural networks are arbitrary, and are known to vary across different random seeds, see tables 5 and 6 in (Chughtai et al., 2023). For the model considered here, we fit the logits to the parameters:

[36] This value was chosen to ensure that the predictive vector of RRMod_p could be encoded in the activations of the residual stream without compression.

$$(\alpha_k)_{k=1,\dots,n} = (3.56, 2.86, 2.42, 2.29), \tag{60}$$
$$(\omega_k)_{k=1,\dots,n} = (38, 30, 41, 18, 36), \tag{61}$$

and achieve a fit of $R^2 = 0.99$.

As indicated in sections 3.2 and 3.3, the predictive vector of both processes RRMod_p and sRRMod_p is non-trivial after observing the token sequences a and ab. In figure 1, we indicate four positions in the residual stream of the transformer that are likely to contain details about the predictive vectors of the generative process. Notable

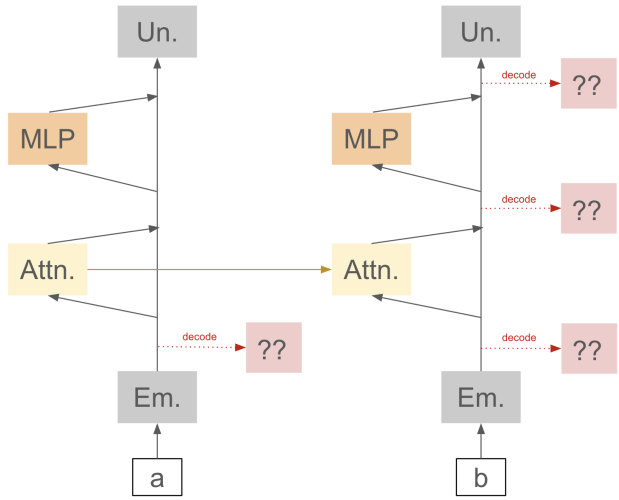

Figure 1: Residual stream locations of the transformer that are likely to contain details of the predictive vector. We use Em. to denote the embedding, Attn. to denote attention, MLP to denote the multi-layer perceptron and Un. to denote the unembedding.

omissions from these locations are the residual stream positions post-attention after observing the token sequence a. As loss is only computed with respect to the final position, i.e., predicting $c = a + b \bmod p$ to follow ab, activations post-attention after observing a single token a do not contribute to predicting the token c, as such, there is no incentive for the model to ensure these activations have any particular structure.

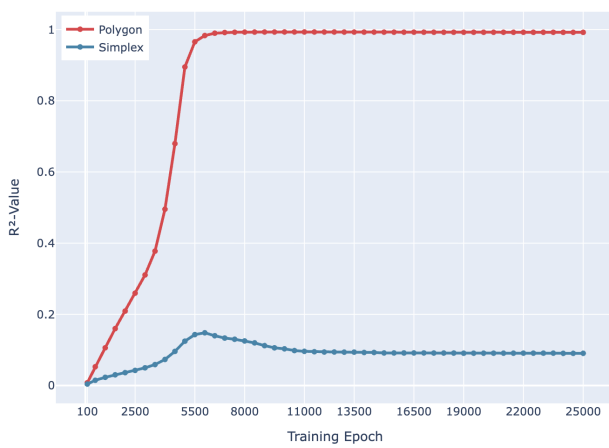

Figure 2: The R^2-value of the linear regression of activations to the vectors $\langle e_p^{(ab)}|$ (Simplex) and $\langle v^{(ab)}|$ (Polygon) evaluated every 500 epochs over the course of training.

Post-MLP pre-unembedding. To evaluate the relevance of both processes to the internal representations of the trained transformer, we perform a linear regression from the activations post-MLP pre-unembedding after observing ab for all $a, b \in \mathcal{X}_p$ to both $\langle e_p^{(ab)}|$ and $\langle v^{(ab)}|$ – the predictive vectors of RRMod_p and sRRMod_p respectively. We find that the fit to $\langle e_p^{(ab)}|$ and $\langle v^{(ab)}|$ achieve R^2-values of 0.09 and 0.99 respectively. This result is stable over training, with the fit to the

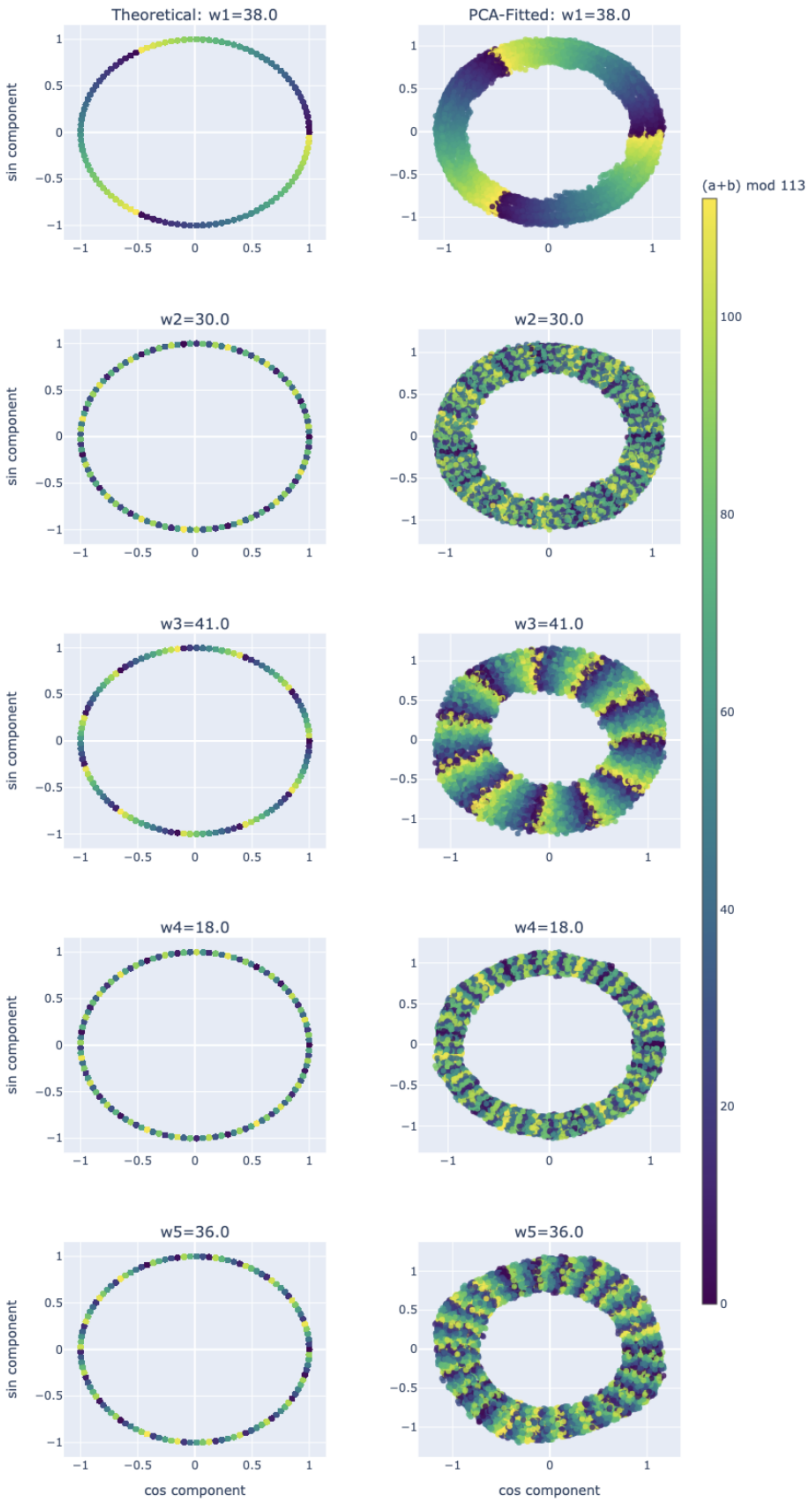

Figure 3: Semi-supervised analysis applied to activations post-MLP pre-unembedding after observing ab. On the left-hand column are the ground-truth predictive vector coordinates $\langle v^{(ab)} |$ and on the right-hand column are the post-processed model activations after observing ab. Each data point corresponds to a sequence ab and are coloured according to the value of $a + b \bmod p$.

vector $\langle v^{(ab)}|$ consistently of higher quality than to $\langle e_p^{(ab)}|$, see figure 2.

Having established the quality of the fit to $\langle v^{(ab)}|$ over $\langle e_p^{(ab)}|$ using supervised methods (linear regression), we support these findings by analysing the same model activations with semi-supervised analysis. We first use principal component analysis[37] (PCA) to reduce the activations to a 10-dimensional vector (consisting of the top 10 PCs), then fit these vectors to (59) using linear regression, where for simplicity we set $(\alpha_k)_{k=1,\ldots,n} = (1,1,1,1,1)$ and use the frequencies (61). The top 10 PCs explain 0.89 of the variance and the linear regression achieves an R^2-value of 0.99. The results of this analysis are presented in the right-hand column of figure 3, where each point corresponds to a processed model-activation after observing ab and whose colour indicates the value of $a + b \bmod p$. In the left-hand column of figure 3, we plot the corresponding predictive vector $\langle v^{(ab)}|$. Comparing the columns for each component, we see a close resemblance between the predictive vectors and model activations. A distinctive qualitative feature of the model activations is the existence of non-trivial *width* to the p-gon, which is plausibly an artifact of how the MLP computes the modular sum, see (Yip et al., 2024) for details.

[37] A linear form of unsupervised analysis.

Post-embedding pre-attention. We continue our analysis by considering activations post-embedding pre-attention after observing a and ab. We perform semi-supervised analysis[38] on the activations at both sequence positions and report the results for the sequence ab, similar results hold for the sequence a. The PCA achieves an explained variance of 0.83 and the linear regression achieves an R^2-value of 0.99. In figure 4, we present plots of the predictive vector $\langle v^{(b)}|$ (left-hand column) and of the processed activations (right-hand column). Unlike for the activations post-MLP pre-unembedding, there is no apparent width to the p-gon for the activations. This observation is consistent with the claim that the MLP causes the width observed in the p-gons in figure 3.

[38] As above, we use PCA then linear regression.

Post-attention pre-MLP. Finally, we consider the activations post-attention pre-MLP after observing the sequence ab. For the activations in this position, we propose they are related to the *sum* of predictive vectors $\langle v^{(a)}| + \langle v^{(b)}|$. As in previous cases, we perform semi-supervised analysis for which the PCA achieves an explained variance of 0.97 and the linear regression achieves an R^2-value of 0.99. We present the results of this analysis in figure 5, where we see qualitative agreement between the predictive vector ansatz and the model activations.

Summary. Having analysed the activations at each of the positions in the transformer suggested in figure 1, we can now fill in the missing

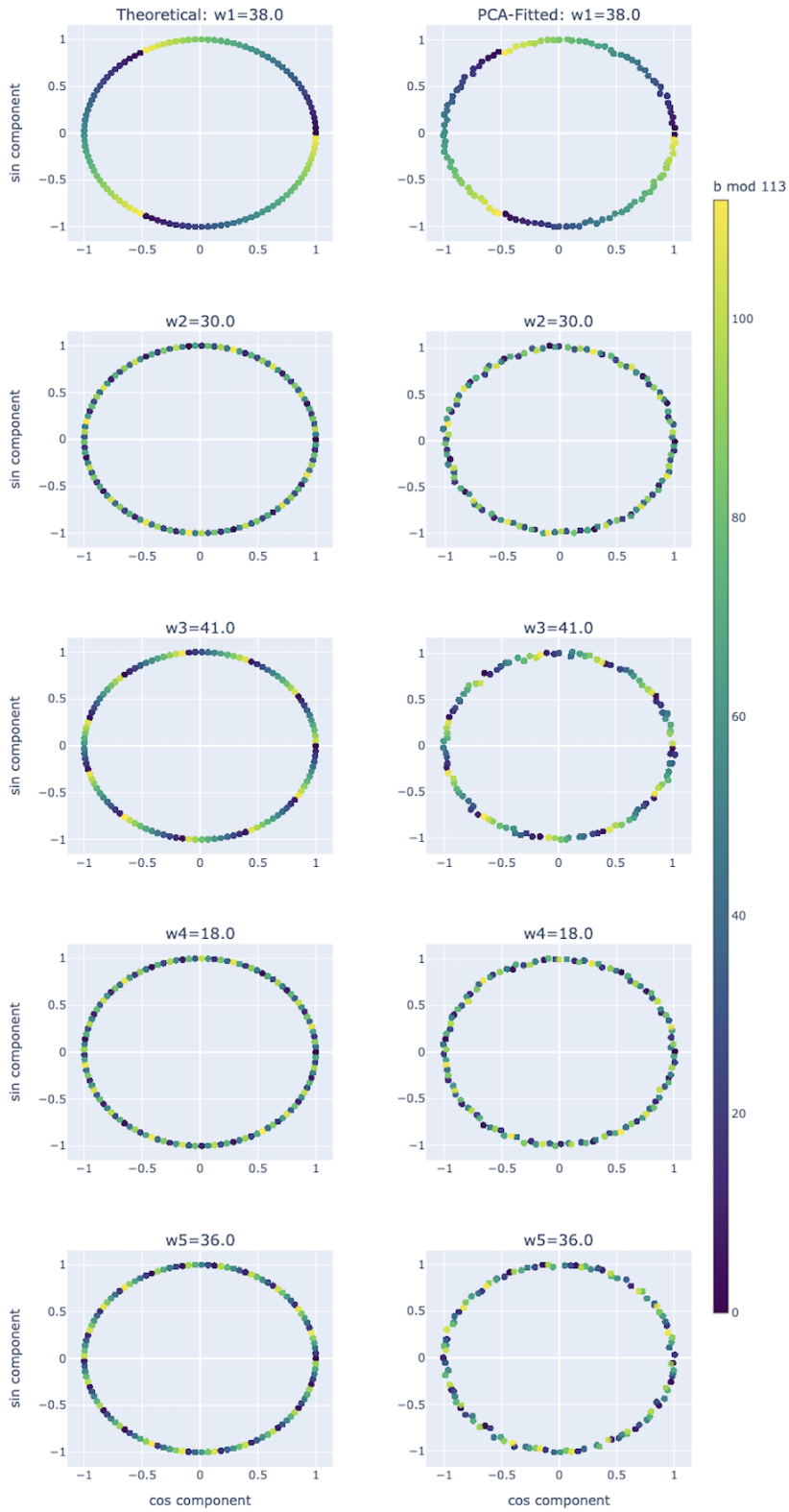

Figure 4: Semi-supervised analysis applied to activations post-embedding pre-attention after observing ab. On the left-hand column are the ground-truth predictive vector coordinates $\langle v^{(b)}|$ and on the right-hand column are the post-processed model activations after observing ab. Each data point corresponds to a sequence ab and are coloured according to the value of b.

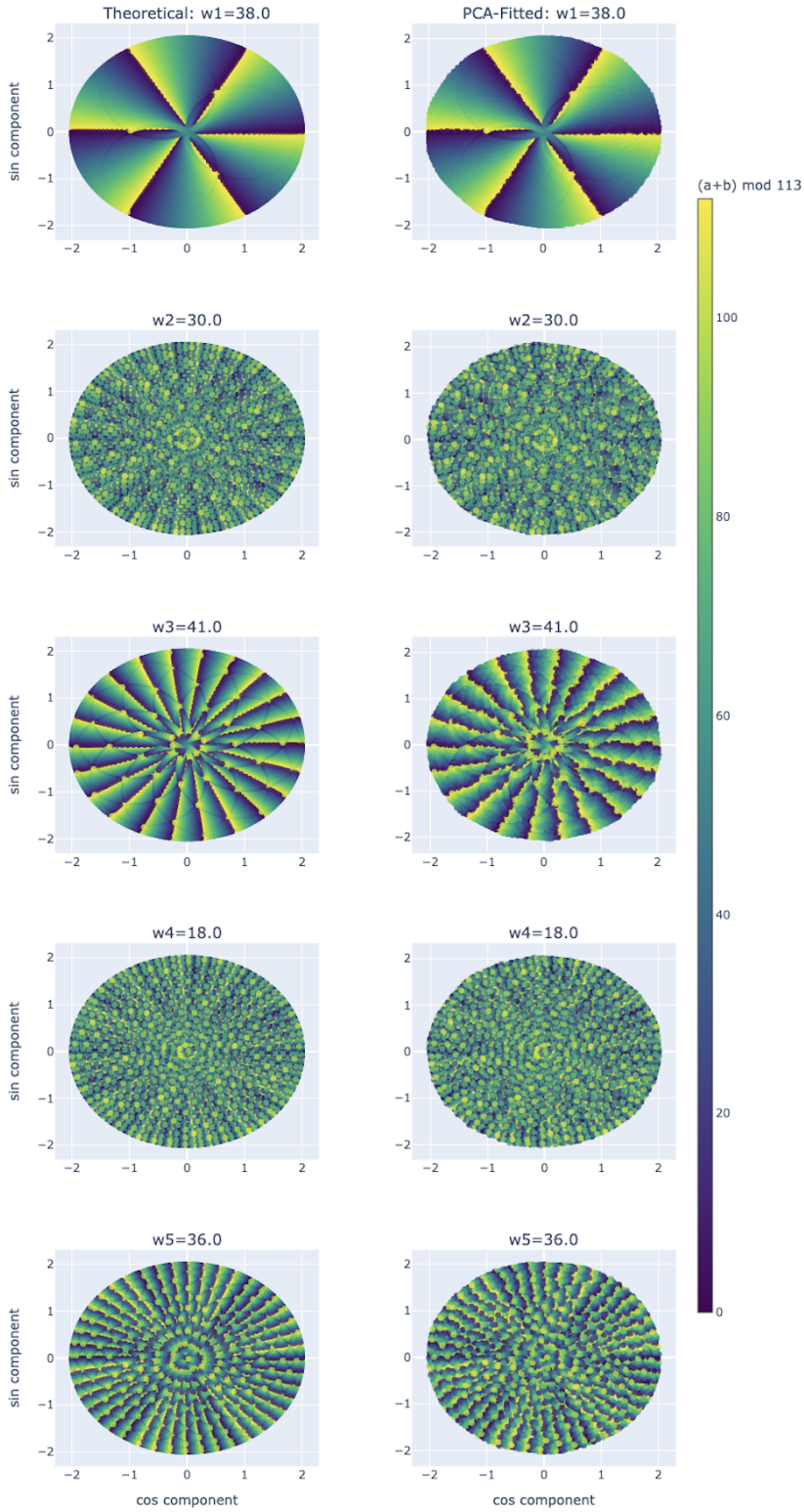

Figure 5: Semi-supervised analysis applied to activations post-attention pre-MLP after observing ab. On the left-hand column are the ground-truth predictive vector coordinates $\langle v^{(a)}| + \langle v^{(b)}|$ and on the right-hand column are the post-processed model activations after observing ab. Each data point corresponds to a sequence ab and are coloured according to the value of $a + b \bmod p$.

details about the object decoded from each position, see figure 6. We summarise the findings here:

- *Post-embedding pre-attention.* The predictive vectors $\langle v^{(a)}|$ and $\langle v^{(b)}|$ are decodable from activations after observations a and ab respectively.
- *Post-attention pre-MLP.* The predictive vector sum $\langle v^{(a)}| + \langle v^{(b)}|$ is decodable from activations after observing ab.
- *Post-MLP pre-unembedding.* The predictive vector $\langle v^{(ab)}|$ is decodable from activations after observing ab.

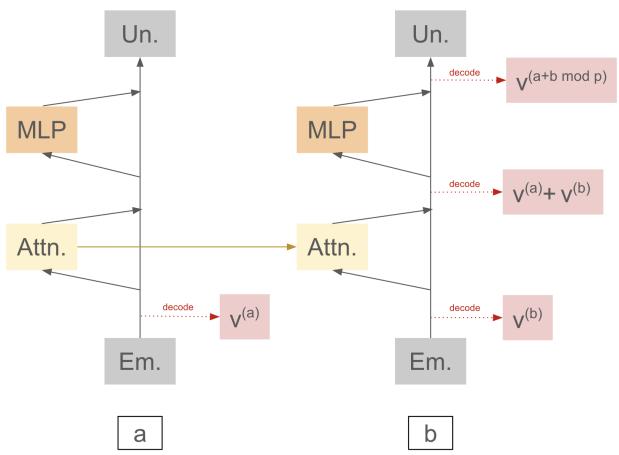

Figure 6: Residual stream locations of the transformer labelled by predictive vectors that are linearly decodable from the model activations.

5 *Discussion*

In section 2, we presented the comp-mech hypothesis, which suggests that for neural networks trained on data from an HMM, the predictive vector of this process is decodable from the network activations. Through sections 3 and 4, we applied this framework to transformers trained to perform modular addition. In this setting, we found that, while the transformer is trained on data from an HMM, its activations admit a linear representation of the predictive vector of an EHMM[39] – a process *approximating* the underlying HMM process. These findings violate the strong form of the comp-mech hypothesis presented in section 2, and suggest the following revised version:

[39] Not the HMM.

Strong comp-mech hypothesis (revised). For neural networks trained on HMM data, the predictive vector *of a process* is linearly decodable from network activations.

While this work has clarified aspects computational mechanics relevant to the internal representations of neural networks, several lim-

itations and open questions remain. We proceed by highlighting directions for future research that address some of these concerns.

An inconvenient aspect of the revised comp-mech hypothesis is that the class of relevant processes is not known, we know from examples, that it must at least include HMMs, GHMMs and the newly introduced EHMMs. To gain insight on details of this class, we suggest applying the computational mechanics framework to other simple algorithmic tasks, for example, those in (Liu et al., 2022).

We studied internal representations of toy-scale transformers performing modular addition, these insights may not translate to large-scale models. While this objection seems plausible, there exists a striking resemblance between the predictive vectors of the sRRMod_p process and the internal representations identified in production-scale transformers performing addition (Kantamneni and Tegmark, 2025). A natural follow-up to (Kantamneni and Tegmark, 2025) is to study modular addition directly and to check whether the internal representations correspond to predictive vectors of sRRMod_p.

Given that predictive vectors of the sRRMod_p process are correlated with the output logits, how can you be sure that it is the *predictive vectors*[40] that the model choses to represent? While this objection applies to the predictive vector decoded from the activations of the transformer post-MLP pre-unembedding, it does not readily apply to those decoded from the activations post-embedding pre-attention, which for token sequences *ab* depend only on *a* or *b*[41]. It is known (Shai et al., 2024), that the predictive vectors of the RRXOR process, when initialised in the stationary distribution, show low correlation with the corresponding next-token probabilities. We anticipate that this will also be true of the RRMod_p[42] and sRRMod_p processes when initialised in the stationary distribution. In this case, we suggest examining the internal representations of transformers trained on sequences[43] from the RRMod_p process and check whether the model continues to represent the predictive vector of sRRMod_p.

[40] As opposed to proto-logits.

[41] Depending on the sequence position from which the activations accessed

[42] Recall that this process is a generalisation of RRXOR.

[43] Possibly sequences longer than three tokens

A Generalised hidden Markov models

A *generalised hidden Markov model* (Upper, 1997; Riechers et al., 2025b) (GHMM) consists of the following data:

- A set $\mathcal{X}$ of *tokens*.
- A collection of *transition matrices* $(T^{(x)})_{x\in\mathcal{X}}$.
- An *initial vector* $\langle\eta^{(\emptyset)}|$.

such that

1. There exists a $|1\rangle$ satisfying:

(a) $T|1\rangle = |1\rangle$ where $T := \sum_{x \in X} T^{(x)}$;

(b) $\langle \eta^{(\emptyset)}|1\rangle = 1$.

2. For all $\ell \in \mathbb{N}$ and for all $x_{1:\ell} \in \mathcal{X}^\ell$, we have

$$\langle \eta^{(\emptyset)}|T^{(x_{1:\ell})}|1\rangle \geq 0. \tag{62}$$

The thrust of this definition, is that the GHMM defines a discrete time stochastic process by assigning probabilities to token sequences via a linear dynamic

$$\Pr(x_{1:\ell}) = \langle \eta^{(\emptyset)}|T^{(x_{1:\ell})}|1\rangle, \qquad T^{(x_{1:\ell})} := T^{(x_1)}T^{(x_2)} \cdots T^{(x_\ell)}. \tag{63}$$

An analogous expression holds for HMMs, see (4). GHMMs differ from HMMs in not making explicit reference to hidden states underlying the stochastic process[44], which impose conditions on the transition matrices, e.g. that they are substochastic. This affords GHMMs the ability to describe a larger class of stochastic processes.

[44] GHMMs are similar to EHMMs in this sense.

B Conditional logits

Proposition B.1. *For all x and for fixed y, the conditional logit $z(y|x)$ can be expressed as*

$$z(y|x) = z(x, y). \tag{64}$$

Proof. Consider

$$\Pr(y|x) = \frac{\Pr(x, y)}{\Pr(x)} = e^{z(x,y)-z(x)} \left[\frac{\sum_{x''} e^{z(x'')}}{\sum_{x',y'} e^{z(x',y')}} \right]. \tag{65}$$

Summing over y, we have

$$\sum_y \Pr(y|x) = \sum_y e^{z(x,y)-z(x)} \left[\frac{\sum_{x''} e^{z(x'')}}{\sum_{x',y'} e^{z(x',y')}} \right] = 1, \tag{66}$$

which implies

$$\frac{\sum_{x''} e^{z(x'')}}{\sum_{x',y'} e^{z(x',y')}} = \frac{1}{\sum_y e^{z(x,y)-z(x)}}. \tag{67}$$

Applying (67) to (65), we have

$$\Pr(y|x) = \frac{e^{z(x,y)-z(x)}}{\sum_y e^{z(x,y)-z(x)}} = \frac{e^{z(x,y)}}{\sum_y e^{z(x,y)}} \tag{68}$$

and can read off that $z(y|x) = z(x, y)$. □

C Representation theory

A *group* G is a set equipped with a binary operation, here denoted by $\circ$, that satisfies:

- *Associativity*. For all $a, b, c \in G$

$$(a \circ b) \circ c = a \circ (b \circ c). \tag{69}$$

- *Identity*. There exists an element $\mathrm{id} \in G$ such that

$$a \circ \mathrm{id} = \mathrm{id} \circ a = a, \tag{70}$$

  for all $a \in G$

- *Inverse*. For each $a \in G$ there exists a $b \in G$ such that

$$a \circ b = b \circ a = \mathrm{id}. \tag{71}$$

A *group representation* is a homomorphism from the group G to a group of invertible matrices $\mathrm{GL}(V)$ for some vector space V. That is,

$$\rho : G \to \mathrm{GL}(V) \tag{72}$$

such that $\rho(gh) = \rho(g)\rho(h)$ for all $g, h \in G$.

A *G-action* describes an action of the group G on a set S in a way that respects the group structure. Formally, it is a function

$$\alpha : S \times G \to S \tag{73}$$

that satisfies:

- *Identity*. For all $s \in S$

$$\alpha(s, \mathrm{id}) = s. \tag{74}$$

- Compatibility. For all $g, h \in G$ and all $s \in S$

$$\alpha(\alpha(s, h), g) = \alpha(s, g \circ h) \tag{75}$$

Each *G-action* induces a *representation* of G

$$\rho_\alpha : G \to \mathrm{GL}(V), \qquad g \mapsto \alpha(\cdot, g) \tag{76}$$

where $V = \mathrm{span}(S)$.

D Transformer architecture details


```
model:
  act_fn: relu
  d_head: 128
  d_mlp: 512
  d_model: 128
  d_vocab: 114
  n_ctx: 3
  n_heads: 1
  n_layers: 1

optimiser:
  betas:
    - 0.9
    - 0.98
  lr: 0.001
  optimizer_type: adamw
  weight_decay: 1.0

training:
  min_delta: 1.0e-06
  num_epochs: 25000
```


References

T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei. Language Models are Few-Shot Learners. 2020. See here.

S. Bubeck, V. Chandrasekaran, R. Eldan, J. Gehrke, E. Horvitz, E. Kamar, P. Lee, Y. T. Lee, Y. Li, S. Lundberg, H. Nori, H. Palangi, M. T. Ribeiro, and Y. Zhang. Sparks of Artificial General Intelligence: Early experiments with GPT-4. 2023. See here.

B. Chughtai, L. Chan, and N. Nanda. A Toy Model of Universality: Reverse Engineering how Networks Learn Group Operations. 2023. See here.

J. P. Crutchfield. Between order and chaos. 2011. See here.

J. P. Crutchfield. Mixed States of Hidden Markov Processes and Their Presentations: What and how to calculate. 2013. See here.

J. P. Crutchfield and D. P. Feldman. Regularities Unseen, Randomness Observed: Levels of Entropy Convergence. 2001. See here.

J. P. Crutchfield and K. Young. Inferring statistical complexity. 1989. See here.

J. P. Crutchfield, C. J. Ellison, and J. R. Mahoney. Time's Barbed Arrow: Irreversibility, Crypticity, and Stored Information. 2009. See here.

H. Cunningham, A. Ewart, L. Riggs, R. Huben, and L. Sharkey. Sparse Autoencoders Find Highly Interpretable Features in Language Models. 2023. See here.

N. Elhage, T. Hume, C. Olsson, and N. Schiefer. Toy Models of Superposition. 2022. See here.

R. Goldstein. Handcrafting a Network to Predict Next Token Probabilities for the Random-Random-XOR Process. 2024. See here.

Y. Jiang, G. Rajendran, P. Ravikumar, B. Aragam, and V. Veitch. On the Origins of Linear Representations in Large Language Models . 2024. See here.

S. Kantamneni and M. Tegmark. Language Models Use Trigonometry to Do Addition. 2025. See here.

J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei. Scaling Laws for Neural Language Models. 2020. See here.

J. Lindsey, W. Gurnee, E. Ameisen, B. Chen, A. Pearce, N. L. Turner, C. Citro, D. Abrahams, S. Carter, B. Hosmer, J. Marcus, M. Sklar, A. Templeton, T. Bricken, C. McDougall, H. Cunningham, T. Henighan, A. Jermyn, A. Jones, A. Persic, Z. Qi, T. B. Thompson, S. Zimmerman, K. Rivoire, T. Conerly, C. Olah, and J. Batson. On the Biology of a Large Language Model. 2025. See here.

B. Liu, J. T. Ash, S. Goel, A. Krishnamurthy, and C. Zhang. Transformers Learn Shortcuts to Automata. 2022. See here.

S. E. Marzen and J. P. Crutchfield. Nearly Maximally Predictive Features and Their Dimensions. 2017. See here.

N. Nanda, L. Chan, T. Lieberum, J. Smith, and J. Steinhardt. Progress measures for grokking via mechanistic interpretability. 2023. See here.

C. Olah. Interpretability Dreams. 2023. See here.

K. Park, Y. Joong Choe, and V. Veitch. The Linear Representation Hypothesis and the Geometry of Large Language Models. 2023. See here.

K. Pepper. RNNs represent belief state geometry in hidden state. 2024. See here.

A. Power, Y. Burda, H. Edwards, I. Babuschkin, and V. Misra. Grokking: Generalization Beyond Overfitting on Small Algorithmic Datasets. 2022. See here.

A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever. Language Models are Unsupervised Multitask Learners. 2019. See here.

P. M. Riechers and J. P. Crutchfield. Spectral Simplicity of Apparent Complexity, Part II: Exact Complexities and Complexity Spectra. 2017. See here.

P. M. Riechers, H. R. Bigelow, E. A. Alt, and A. Shai. Next-token pretraining implies in-context learning. 2025a. See here.

P. M. Riechers, T. J. Elliott, and A. S. Shai. Neural networks leverage nominally quantum and post-quantum representations. 2025b. See here.

A. S. Shai, S. E. Marzen, L. Teixeira, A. G. Oldenziel, and P. M. Riechers. Transformers represent belief state geometry in their residual stream. 2024. See here.

C. R. Shalizi and J. P. Crutchfield. Computational Mechanics: Pattern and Prediction, Structure and Simplicity. 1999. See here.

S. Still, J. P. Crutchfield, and C. J. Ellison. Optimal Causal Inference: Estimating Stored Information and Approximating Causal Architecture. 2007. See here.

A. Templeton, T. Conerly, J. Marcus, J. Lindsey, T. Bricken, B. Chen, A. Pearce, C. Citro, E. Ameisen, A. Jones, H. Cunningham, N. L. Turner, C. McDougall, M. MacDiarmid, A. Tamkin, E. Durmus, T. Hume, F. Mosconi, C. D. Freeman, T. R. Sumers, E. Rees, J. Batson, A. Jermyn, S. Carter, C. Olah, and T. Henighan. Scaling Monosemanticity: Extracting Interpretable Features from Claude 3 Sonnet. 2024. See here.

D. R. Upper. Theory and algorithms for hidden Markov models and generalized hidden Markov models . 1997. See here.

W. Wu, L. Jaburi, J. Drori, and J. Gross. Towards a unified and verified understanding of group-operation networks. 2024. See here.

C. H. Yip, R. Agrawal, L. Chan, and J. Gross. Modular addition without black-boxes: Compressing explanations of MLPs that compute numerical integration. 2024. See here.